Laboratorium Sztucznej Inteligencji

Klasyfikator Bayesa

Mateusz Braszczok, Daniel Kopiecki

21 grudnia 2022



Politechnika Śląska

Contents

1 Zadanie 1 **3**
1.1 Metoda parametryczna w oparciu o rozkład normalny 3
1.2 Metoda nieparametryczna w oparciu o okna Parzena 4

2 Zadanie 3 **4**
2.1 Badanie wpływu szerokości okna na uzyskane rezultaty 6

3 Zadanie 4 **7**

4 Wnioski **10**

5 Kod źródłowy **10**

List of Tables

List of Figures

1 Wygenerowane dane 3
2 Rozkłady prawdopodobieństwa dla wygenerowanych danych 3
3 Przeprowadzona klasyfikacja 5
4 Rozkład cech dla klas A1 i A2, okno h_1 6
5 Rozkład cech dla klas A1 i A2, okno h_2 6
6 Rozkład cech dla klas A1 i A2, okno h_3 7
7 Wygenerowane dane 7
8 Uzyskane rozkłady (lewy parametryczny) 8
9 Uzyskane rozkłady (lewy parametryczny) 8
10 Przykładowy przebieg klasyfikacji (dla metody parametrycznej) 8
11 Wygenerowane dane 9
12 Uzyskane rozkłady (lewy parametryczny) 9
13 Uzyskane rozkłady (lewy parametryczny) 9
14 Przykładowy przebieg klasyfikacji (dla metody parametrycznej) 10

1 Zadanie 1

Wygenerowano syntetyczny zbiór składający się z 200 próbek, spośród których można rozróżnić dwie klasy abstrakcji, a każda próbka ma zestaw dwóch cech. Klasy A1 i A2 zostały wygenerowane z rozkładu normalnego o parametrach:

$$\mu_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \sigma_1 = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} \tag{1}$$

$$\mu_2 = \begin{bmatrix} 2 \\ 2 \end{bmatrix}, \quad \sigma_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \tag{2}$$

Dane zostały następnie w sposób losowy podzielone na zbiór uczący i testowy, gdzie każdy z nich liczył po 100 próbek. Efekt tego działania przedstawiono na rysunku poniżej:

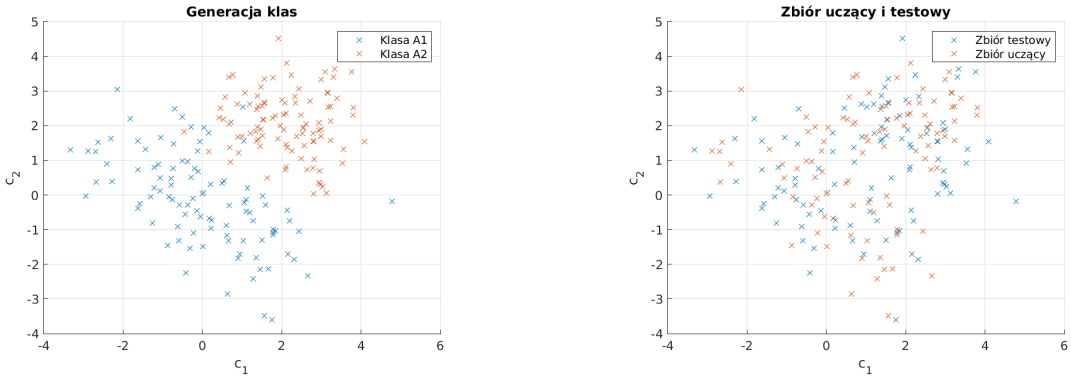


Figure 1: Wygenerowane dane

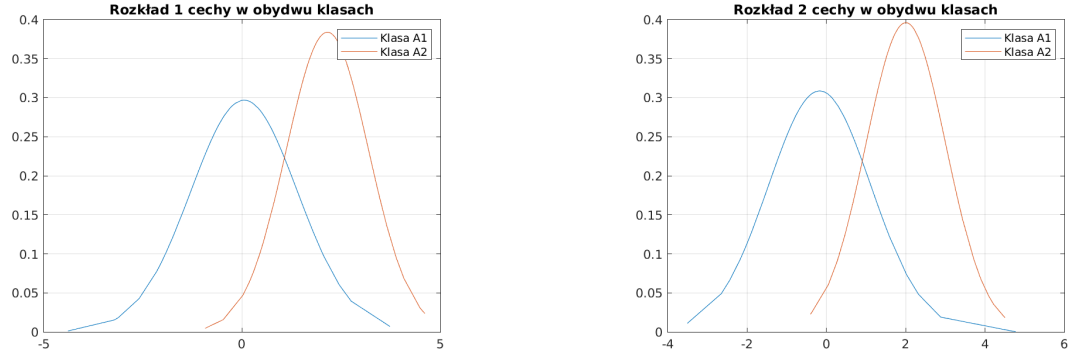


Figure 2: Rozkłady prawdopodobieństwa dla wygenerowanych danych

Zaimplementowano funkcję klasyfikatora Bayes'a dla 2-klasowego zbioru danych i dowolnej liczby cech. Do estymacji rozkładu wykorzystano dwie metody.

1.1 Metoda parametryczna w oparciu o rozkład normalny

Gęstość prawdopodobieństwa wystąpienia próbki w danej klasie jest określana na podstawie rozkładu normalnego. Argumentami zaimplementowanej funkcji w tym przypadku jest μ i σ, czyli odpowiednio średnia oraz odchylenie standardowe danej klasy. Implementacja

w skrypcie m jest w tym przypadku trywialna, ponieważ działa ona w oparciu o funkcję *normpdf*, zwracającą gęstość prawdopodobieństwa z rozkładu normalnego, oraz funkcję *sum*, która jak sama nazwa wskazuje, sumuje uzyskane gęstości przemnożone przez z góry określone prawdopodobieństwo wystąpienia danej klasy. Jej implementację przedstawiono na poniższym listingu.

```
function prob = bayes(obj, x, mu, sigma, w)
    distr = normpdf(x, mu, sigma)
    prob = 0;
    for i = 1 : obj.NUM_OF_CLASES
        for j = 1 : length(distr)
            prob = prob + distr(j) * w(i);
        end
    end
end
```

1.2 Metoda nieparametryczna w oparciu o okna Parzena

Metoda ta dopasowuje funkcje gęstości prawdopodobieństwa do przekazanych próbek. Wykorzystano funkcje *fitdist* i *pdf*, które na podstawie przekazanych próbek oraz długości okna w kombinacji zwracają prawdopodobieństwo wystąpienia próbki w danej klasie. Obliczona funkcja gęstości prawdopodobieństwa jest następnie wykorzystana do procedury klasyfikacji, która jest analogiczna jak w przypadku metody parametrycznej w oparciu o rozkład normalny. Implementacje przedstawiono na poniższym listingu.

```
function distr = getParzenDistr(obj, set, windowH)
    pd = fitdist(set(:, 1), "Kernel", "Kernel", "box", Width=
        windowH);
    x = linspace(min(set), max(set), (obj.N * 2) + 1);
    y = pdf(pd, x);
    parzen = [x; y];
end

function prob = parzen(obj, set, distr, w)
    prob = 0;

    for i = 1 : length(w)
        [~, x] = min(abs(distr(1, :, i) - set(i)));
        traitProb = distr(2, x, i);
        prob = prob + traitProb * w(i);
    end
end
```

2 Zadanie 3

Dla zaimplementowanego klasyfikatora Bayes'a i dwóch metod estymacji rozkładu w danych klasach wyznaczono jakość klasyfikacji w oparciu o następującą macierz błędów:

- TP - prawdziwie pozytywna
- TN - prawdziwie negatywna
- FP - fałszywie pozytywna
- FN - fałszywie negatywna

Obliczona macierz błędów posłuży następnie do wyznaczenia wskaźnika jakości klasyfikacji danego następującą zależnością:

$$ACC = \frac{TP + TN}{TP + FP + TN + FN} \tag{3}$$

Dla każdej metody przeprowadzono 10 klasyfikacji dla dwóch klas danych z dwoma cechami, a okno długość okna Parzena wynosiła $h = 0.5$.

Numer eksperymentu	Gauss, ACC [%]	Parzen, ACC [%]
1	**86**	84
2	80	80
3	**82**	81
4	83	**84**
5	75	**77**
6	81	81
7	**83**	81
8	86	86
9	**83**	82
10	80	**81**

Jak można wywnioskować z zebranych wyników, obydwie metody dają dość podobne rezultaty. Średnia wskaźnika ACC w przeprowadzonych eksperymentach wynosiła odpowiednio:

- Dla metody parametrycznej - $ACC_m = 81.9\%$
- Dla okien Parzena - $ACC_m = 81.7\%$

Oznacza to, że wykorzystanie metody parametrycznej do generowania rozkładu daje znikomo lepsze rezultaty, dla tak dobranej szerokości okna. Przykładowe graficzne zobrazowanie dla przeprowadzonych klasyfikacji przedstawiono na rysunku poniżej.

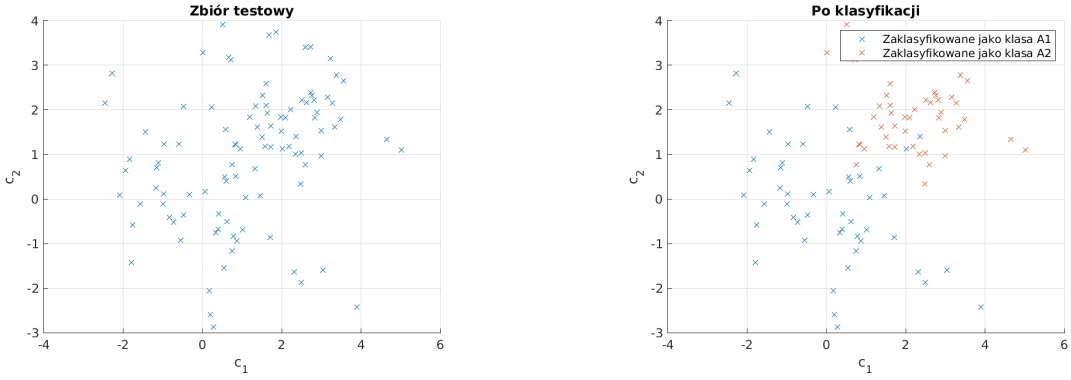


Figure 3: Przeprowadzona klasyfikacja

2.1 Badanie wpływu szerokości okna na uzyskane rezultaty

Aby zbadać wpływ szerokości okna, wybrano trzy wartości jego długości które wynosiły odpowiednio:

1. $h_1 = 1$
2. $h_2 = 2$
3. $h_3 = 3$

Dla każdej szerokości przeprowadzono następnie trzy klasyfikacje, których wyniki są następujące:

Numer eksperymentu	ACC dla h_1 [%]	ACC dla h_2 [%]	ACC dla h_3 [%]
1	84	70	50
2	84	76	65
3	83	76	55

Jak można wywnioskować z uzyskanych danych, wraz ze wzrostem długości okna, jakość klasyfikacji spada. Przykładowe rozkłady uzyskane dla sprawdzonych długości okien można zaobserwować na rysunkach poniżej.

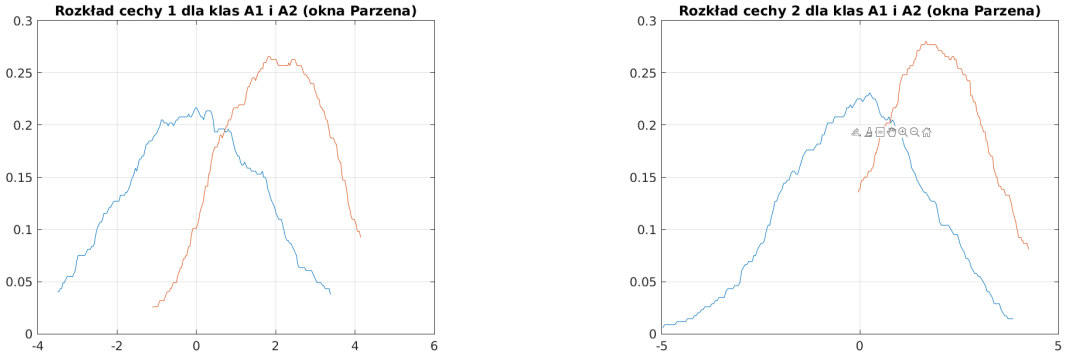


Figure 4: Rozkład cech dla klas A1 i A2, okno h_1

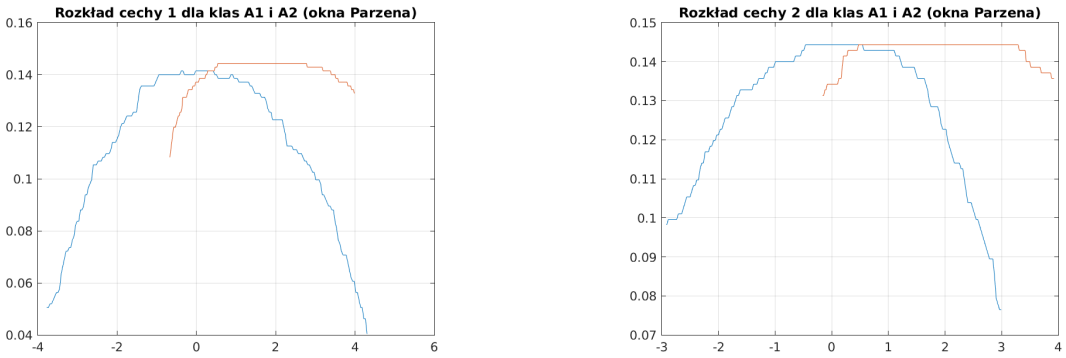


Figure 5: Rozkład cech dla klas A1 i A2, okno h_2

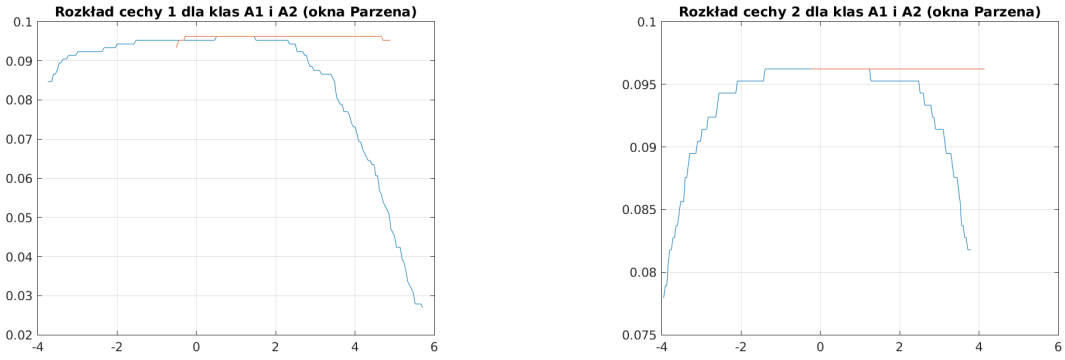


Figure 6: Rozkład cech dla klas A1 i A2, okno h_3

3 Zadanie 4

W celu dalszej analizy autorzy postanowili sprawdzić, jak dwie wykorzystane metody zachowają się w przypadku, kiedy wygenerowane dane będą pochodziły z innych rozkładów niż normalny. W tym celu działanie klasyfikatora przetestowano dla rozkładów:

1. Trójkątny, gdzie:
$$a_1 = 2, \quad b_1 = 5, \quad c_1 = 6 \tag{4}$$
$$a_2 = 3, \quad b_2 = 6, \quad c_2 = 7 \tag{5}$$
2. Chi kwadrat, gdzie:
$$\nu_1 = 10, \quad \nu_2 = 22 \tag{6}$$

Dla rozkładu chi kwadrat uzyskano następujące rezultaty:

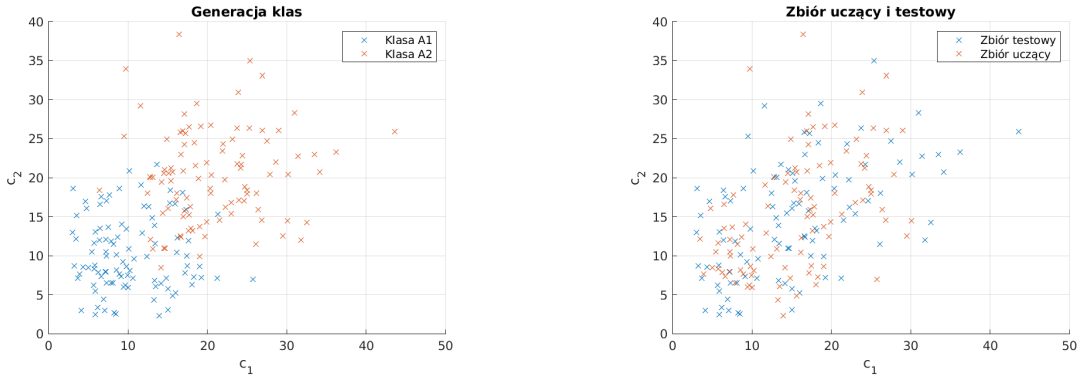


Figure 7: Wygenerowane dane

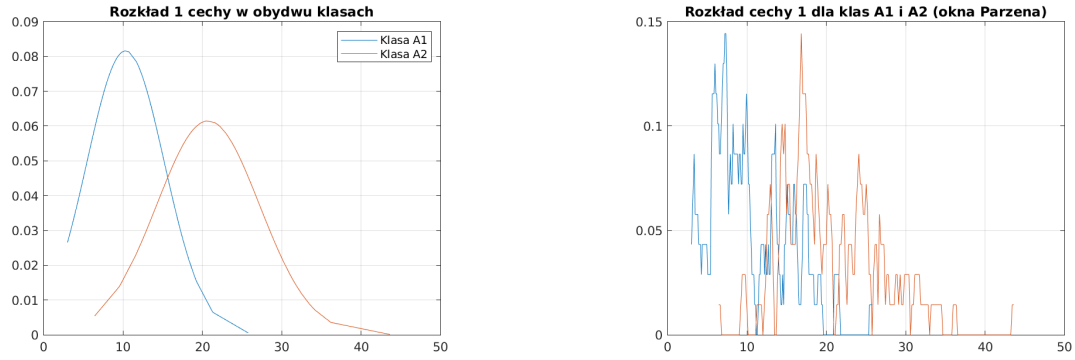


Figure 8: Uzyskane rozkłady (lewy parametryczny)

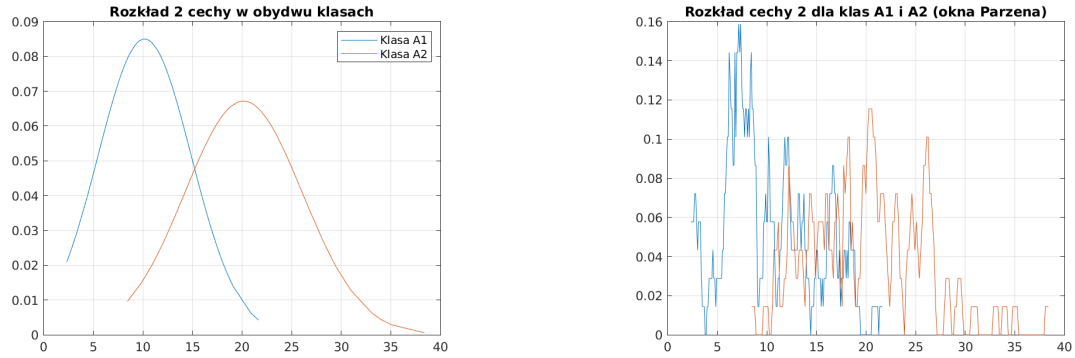


Figure 9: Uzyskane rozkłady (lewy parametryczny)

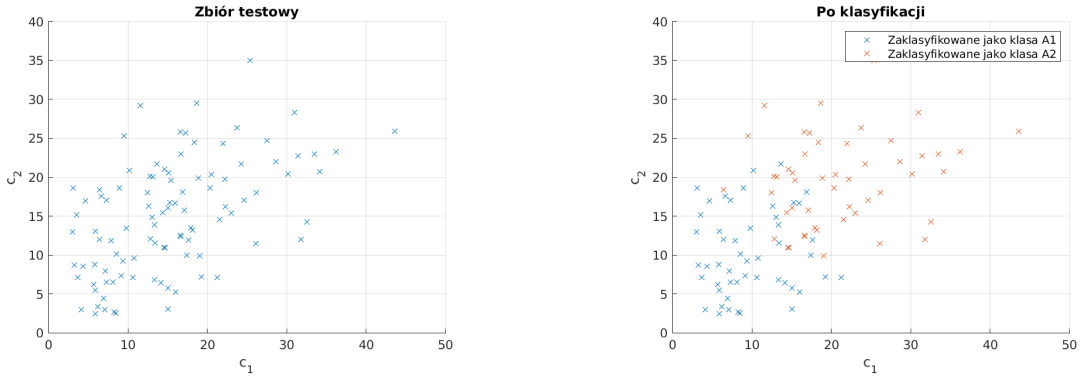


Figure 10: Przykładowy przebieg klasyfikacji (dla metody parametrycznej)

Porównywując wskaźnik ACC dla obu metod autorzy zauważyli, że przy każdym nowym zestawie danych, metoda okien Parzena dawała lepszą jakość klasyfikacji (dla dziesięciu eksperymentów średnia wartość wskaźnika dla metody parametrycznej wynosiła 89%, gdzie dla metody okien Parzena 92%).

Dla rozkładu trójkątnego uzyskano następujące rezultaty:

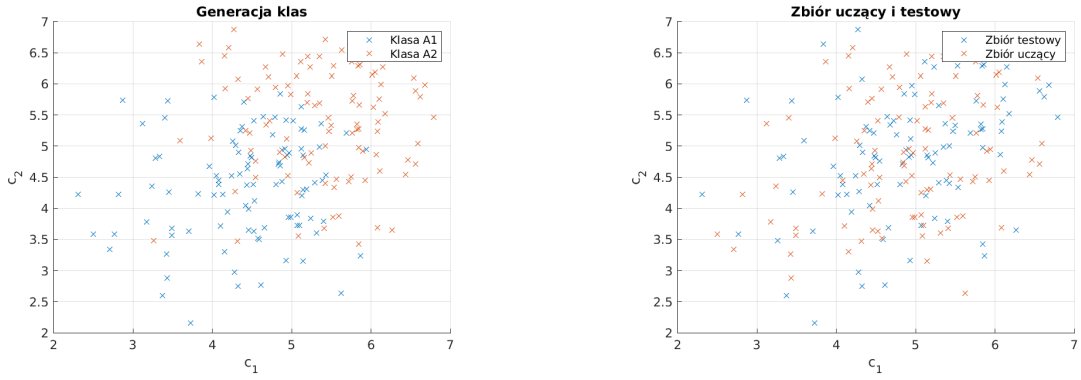


Figure 11: Wygenerowane dane

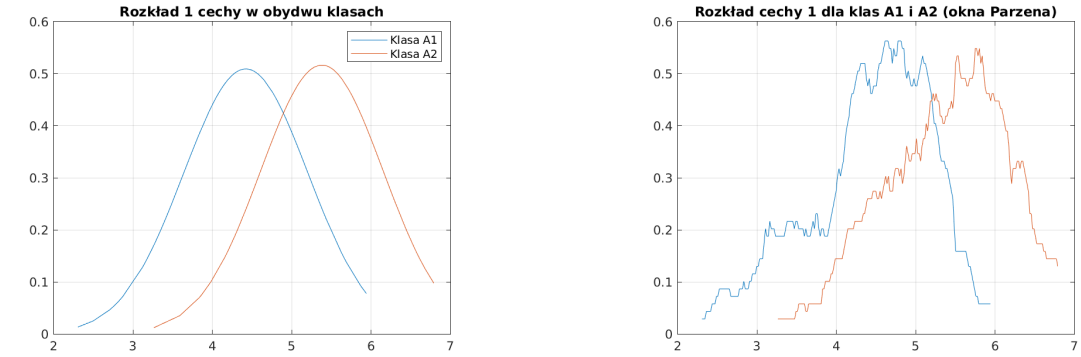


Figure 12: Uzyskane rozkłady (lewy parametryczny)

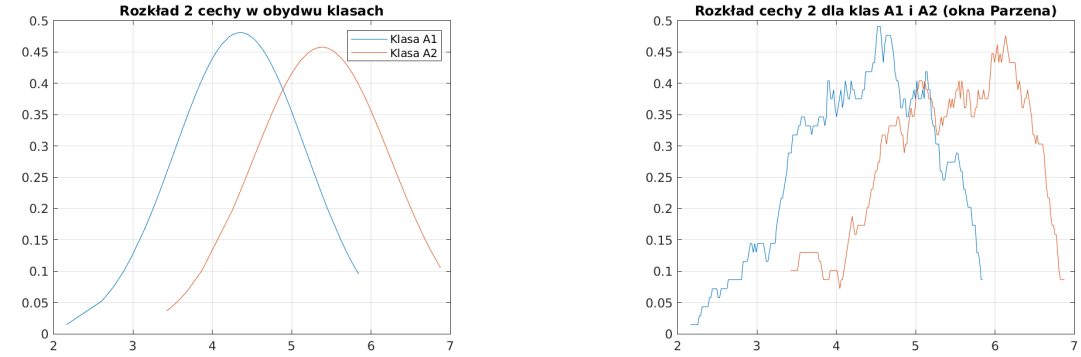


Figure 13: Uzyskane rozkłady (lewy parametryczny)

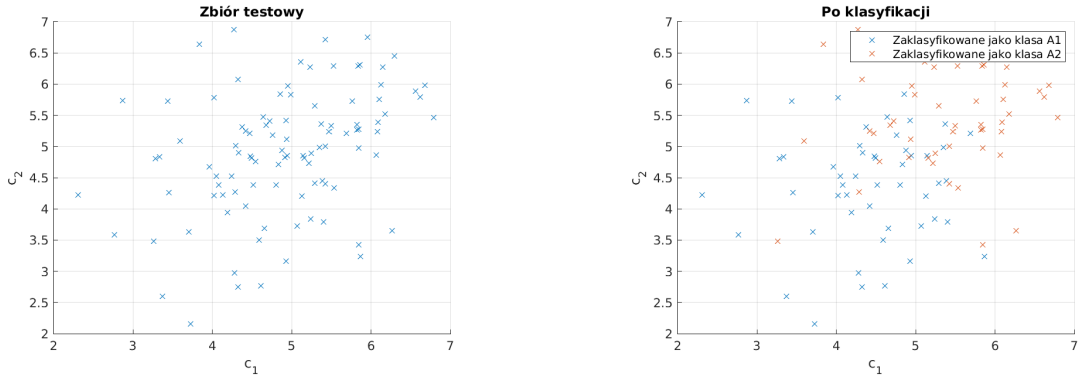


Figure 14: Przykładowy przebieg klasyfikacji (dla metody parametrycznej)

W tym przypadku analizując wskaźniki ACC dla obu metod autorzy zauważyli, że metoda okien Parzena daje jeszcze lepsze rezultaty (dla dziesięciu eksperymentów średnia wartość wskaźnika dla metody parametrycznej wynosiła 72%, gdzie dla metody okien Parzena 78%).

4 Wnioski

W przypadku, kiedy dane mają rozkład normalny, metoda parametryczna daje znikomo lepsze rezultaty. Powodem jest dobre dopasowanie rozkładów, które w obu przypadkach mają ten sam charakter. Kiedy jednak rozkład klasyfikowanych danych jest inny, metoda okien Parzena okazuje się bardziej odpowiednia, ponieważ nie zakłada z góry określone rozkładu, tylko go dopasowuje. Implikuje to lepszą uniwersalność metody Parzena w przypadku, kiedy go nie znamy.

Niestety autorom nie udało się sprawdzić jakości klasyfikacji dla zbiorów danych o liczbie cech większej niż 2. W przypadku, kiedy na wejście algorytmów przekazywano takie dane, dokładność była bardzo znikoma, więc zrezygnowano z tej części zadania.

5 Kod źródłowy

```
1  classdef SI1
2      properties
3          firstClassMu
4          firstClassSigma
5          secondClassMu
6          secondClassSigma
7          numOfTraits
8
9          A1
10         A1_distr
11
12         A2
13         A2_distr
14
15         testSet
```

```
16          trainingSet
17
18          acc
19
20          PARAMETERIC = 0
21          PARZEN = 1
22          N = 100
23          NORMAL = 0
24          TRIANGULAR = 1
25          CHI = 2
26      end
27
28      methods (Access = private)
29          function distr = getDistr(obj, set)
30              mu = mean(set);
31              sigma = std(set);
32              pd = normpdf(sort(set), mu, sigma);
33
34              distr = Distr(mu, sigma, pd);
35          end
36
37          function parzen = getParzenDistr(obj, set, windowH)
38              pd = fitdist(set(:, 1), "Kernel", "Kernel", "box",
                    Width=windowH);
39              x = linspace(min(set), max(set), (obj.N * 2) + 1);
40              y = pdf(pd, x);
41              parzen = [x; y];
42          end
43
44          function [firstClassDistr, secondClassDistr] =
                getParzenFullDistr(obj, windowH, class1, class2,
                numOfTraits)
45              for i = 1 : numOfTraits
46                  firstClassDistr(:, :, i) = obj.getParzenDistr(
                        class1(:, i), windowH);
47                  secondClassDistr(:, :, i) = obj.getParzenDistr(
                        class2(:, i), windowH);
48              end
49          end
50
51          function prob = bayes(obj, x, mu, sigma, w)
52              distr = normpdf(x, mu, sigma);
53              prob = 0;
54              for i = 1 : 2
55                  for j = 1 : length(distr)
56                      prob = prob + distr(j) * w(i);
57                  end
58              end
59          end
```

```
60
61          function prob = parzen(obj, set, distr, w)
62              prob = 0;
63
64              for i = 1 : length(w)
65                  [~, x] = min(abs(distr(1, :, i) - set(i)));
66                  traitProb = distr(2, x, i);
67                  prob = prob + traitProb * w(i);
68              end
69          end
70      end
71
72      methods (Access = public)
73          function obj = SI1(firstClassMu, firstClassSigma,
                secondClassMu, secondClassSigma, numOfTraits, type)
74              if (type == obj.NORMAL)
75                  obj.A1 = mvnrnd(firstClassMu, firstClassSigma,
                        obj.N);
76                  obj.A2 = mvnrnd(secondClassMu, secondClassSigma,
                        obj.N);
77              elseif (type == obj.TRIANGULAR)
78                  pd = makedist('Triangular', 2, 5, 6);
79                  obj.A1 = random(pd, obj.N, 2);
80
81                  pd = makedist('Triangular', 3, 6, 7);
82                  obj.A2 = random(pd, obj.N, 2);
83
84              elseif (type == obj.CHI)
85                  nu = 10;
86                  obj.A1 = chi2rnd(nu, obj.N, 2);
87                  obj.A2 = chi2rnd(nu + 12, obj.N, 2);
88              end
89
90              obj.numOfTraits = numOfTraits;
91
92              gcf = figure("visible", "off");
93              scatter(obj.A1(:, 1), obj.A1(:, 2), 'x');
94              grid on;
95              hold on;
96              scatter(obj.A2(:, 1), obj.A2(:, 2), 'x');
97              title("Generacja klas");
98              xlabel("c_1");
99              ylabel("c_2");
100             legend("Klasa A1", "Klasa A2");
101             saveas(gcf, "images/a1_a2.png")
102
103             obj.testSet(1:50, 1:2) = obj.A1(1:50, :);
104             obj.testSet(1:50, 3) = ones(50, 1);
105             obj.trainingSet(1:50, :) = obj.A2(1:50, :);
```

```
106
107             obj.testSet(51:100, 1:2) = obj.A2(51:100, :);
108             obj.testSet(51:100, 3) = 2*ones(50, 1);
109             obj.trainingSet(51:100, :) = obj.A1(51:100, :);
110
111             gcf = figure("visible", "off");
112             scatter(obj.testSet(:, 1), obj.testSet(:, 2), "x");
113             hold on;
114             scatter(obj.trainingSet(:, 1), obj.trainingSet(:, 2),
                    "x");
115             grid on;
116             title("Zboor uczacy i testowy");
117             xlabel("c_1");
118             ylabel("c_2");
119             legend("Zbior testowy", "Zbior uczacy");
120             saveas(gcf, "images/test_training.png");
121
122         end
123
124         function acc = zad2(obj, type)
125             w = [0.5, 0.5];
126             obj.acc = ACC();
127             mu = [];
128             sigma = [];
129
130             if (type == obj.PARAMETERIC)
131                 for i = 1 : obj.numOfTraits
132                     obj.A1_distr{i} = obj.getDistr(obj.A1(:, i));
133                     obj.A2_distr{i} = obj.getDistr(obj.A2(:, i));
134
135                     gcf = figure("visible", "off");
136                     plot(sort(obj.A1(:, i)), obj.A1_distr{i}.pd);
137                     hold on;
138                     plot(sort(obj.A2(:, i)), obj.A2_distr{i}.pd);
139                     grid on;
140                     title("Rozklad " + num2str(i) + " cechy w
                            obydwu klasach")
141                     legend("Klasa A1", "Klasa A2")
142                     saveas(gcf, "images/a12_" + num2str(i) + "
                            _distr.png");
143
144                     mu1(i) = obj.A1_distr{i}.mu;
145                     sigma1(i) = obj.A1_distr{i}.sigma;
146
147                     mu2(i) = obj.A2_distr{i}.mu;
148                     sigma2(i) = obj.A2_distr{i}.sigma;
149                 end
150
151                 gcf = figure("visible", "off");
```

```
152                 scatter(obj.testSet(:, 1), obj.testSet(:, 2), "x
                        ");
153                 grid on;
154                 title("Zbior testowy");
155                 xlabel("c_1");
156                 ylabel("c_2");
157                 saveas(gcf, "images/test.png");
158
159                 afterClassification_A1 = [];
160                 afterClassification_A2 = [];
161                 j = 1;
162                 k = 1;
163
164                 for i = 1 : obj.N
165                     testPoint = obj.testSet(i, 1 : obj.
                            numOfTraits);
166
167                     p(1) = obj.bayes(testPoint, mu1, sigma1, w);
168                     p(2) = obj.bayes(testPoint, mu2, sigma2, w);
169
170                     testPoint = obj.testSet(i, :);
171                     obj.acc.calculate(p, testPoint);
172                     if (testPoint(:, 3) == 1)
173                         afterClassification_A1(j, :) = testPoint;
174                         j = j + 1;
175                     else
176                         afterClassification_A2(k, :) = testPoint;
177                         k = k + 1;
178                     end
179
180                 end
181
182                 disp("hi")
183                 size(afterClassification_A1)
184                 size(afterClassification_A2)
185
186                 gcf = figure("visible", "off");
187                 scatter(afterClassification_A1(:, 1),
                        afterClassification_A1(:, 2), "x");
188                 hold on;
189                 scatter(afterClassification_A2(:, 1),
                        afterClassification_A2(:, 2), "x");
190                 grid on;
191                 title("Po klasyfikacji");
192                 xlabel("c_1");
193                 ylabel("c_2");
194                 legend("Zaklasyfikowane jako klasa A1", "
                        Zaklasyfikowane jako klasa A2");
195                 saveas(gcf, "images/class.png");
```

```
196
197             elseif (type == obj.PARZEN)
198                 windowH = 0.2;
199                 [firstClassDistr, secondClassDistr] = obj.
                        getParzenFullDistr(windowH, obj.A1, obj.A2,
                        obj.numOfTraits);
200
201                 for i = 1 : obj.numOfTraits
202                     gcf = figure("visible", "off");
203                     plot(firstClassDistr(1, :, i)',
                            firstClassDistr(2, :, i)');
204                     hold on
205                     plot(secondClassDistr(1, :, i)',
                            secondClassDistr(2, :, i)');
206                     hold off;
207                     title("Rozklad cechy " + num2str(i) + " dla
                            klas A1 i A2 (okna Parzena)");
208                     grid on;
209                     saveas(gcf, "images/parzen_" + num2str(i) + "
                            _" + num2str(windowH) + ".png");
210                 end
211
212                 for i = 1 : obj.N
213                     testPoint = obj.testSet(i, 1 : obj.
                            numOfTraits);
214
215                     p(1) = obj.parzen(testPoint, firstClassDistr,
                             w);
216                     p(2) = obj.parzen(testPoint, secondClassDistr
                            , w);
217
218                     testPoint = obj.testSet(i, :);
219
220                     obj.acc.calculate(p, testPoint);
221                 end
222
223             end
224
225             acc = obj.acc.acc;
226         end
227
228         function comparison(obj)
229
230             [firstClassDistr, secondClassDistr] = obj.
                    getParzenFullDistr(windowH, obj.A1, obj.A2, obj.
                    NUM_OF_TRAITS);
231
232             firstClassFirstTraitDistr = firstClassDistr(:, :, 1);
```

```
            firstClassSecondTraitDistr = firstClassDistr(:, :, 2)
                ;

            secondClassFirstTraitDistr = secondClassDistr(:, :,
                1);
            secondClassSecondTraitDistr = secondClassDistr(:, :,
                2);

        end
    end

end
```

```
classdef ACC < handle
    properties
        tp
        tn
        fp
        fn
        acc
    end

    methods
        function obj = ACC()
            obj.tp = 0;
            obj.tn = 0;
            obj.fp = 0;
            obj.fn = 0;
            obj.acc = 0;
        end

        function calculate(obj, prob, test)
            p1 = prob(1);
            p2 = prob(2);

            if (test(:, 3) == 2)
                if (p1 > p2)
                    obj.fp = obj.fp + 1;
                else
                    obj.tn = obj.tn + 1;
                end
            else
                if (p1 > p2)
                    obj.tp = obj.tp + 1;
                else
                    obj.fn = obj.fn + 1;
                end
            end
            obj.acc = (obj.tp + obj.tn) / (obj.tp + obj.tn + obj.
```

```
                fp + obj.fn) * 100;
        end
    end
end
```

```
classdef Distr
    properties
        sigma
        mu
        pd
    end

    methods
        function obj = Distr(mu, sigma, pd)
            obj.sigma = sigma;
            obj.mu = mu;
            obj.pd = pd;
        end
    end
end
```